OVLA: On-device Verification of LLM-Generated IoT Automations via Timeline IR

Anonymous Author(s)

Abstract

Smart home automation increasingly relies on large language models (LLMs) to translate natural language (NL) user intents into executable reactive rules. However, deploying these rules directly on edge hubs poses severe reliability and safety risks since reactive IoT automations are temporally complex, involving precise triggers, durations, and repetitions. Existing validation methods rely heavily on human code inspection, offline experts, or resource-heavy cloud LLM re-checks, and thus fail to meet edge privacy and latency constraints. To tackle this challenge, we present OVLA, an on-device pipeline for deterministic, LLM-free verification of IoT automations before deployment. OVLA bridges the gap between informal natural language and strict execution with Timeline IR, a compact, machine-checkable representation of temporal behavior: the user's intent is translated into a typed Timeline IR, and the user confirms its deterministic plain-language rendering, which then serves as the reference specification. OVLA then synthesizes boundary-event scenarios from this IR, co-simulates the IR and the generated rule code, and compares their action traces to detect behavioral discrepancies and guide automated repairs via counterexamples. On a benchmark of 382 natural-language automations, the verifier caught 99.3% of 1,552 injected code faults. When the full pipeline ran end-to-end with local quantized ≤9B generators, the gate flagged 35 candidates; 11 were repaired and 24 rejected fail-closed, and no divergent candidate reached deployment; 35,800 randomized dense replays independently confirmed the conformance of the final deployed set. The verdict is available on the device at deployment time: the LLM-free gate decides in 0.97ms at the median on a Mac mini M4, and we integrate OVLA end-to-end with a commercial home-automation platform and physical devices.

Keywords

IoT automation, LLM code generation, reactive-temporal code, deterministic verification, on-device, trace equivalence

1 Introduction

IoT automation is rapidly moving toward LLM-based code generation. A user states the desired behavior in natural language, and an LLM turns it into a reactive rule the platform can execute. Yet the final step of this workflow, deciding *whether the generated code is correct to deploy*, has no deterministic gate. The missing piece is a reference to check the code against; the natural-language request contains the intent, but not in a form that can serve as a rigorous baseline for automated testing.

Why this is hard: The core challenge stems from the absence of a verifiable reference baseline. The target automations are inherently *reactive-temporal*. They are *reactive* because their execution is driven by asynchronous events and conditional triggers, and *temporal* because their behavior unfolds over time through schedules, periodic executions, and sustained state durations (e.g. "if it persists for more than N minutes"). Driven by timers, counters, and multi-step state, the correct behavior of an automation must be evaluated across an entire timeline rather than a single discrete instant. Furthermore, in our setting, this logic is deployed as arbitrary code rather than a restricted, fixed-rule template. Validating their correctness presents two key challenges. First, no test oracle is available. Unlike text-to-SQL or traditional programming benchmarks that supply gold denotations or input-output test suites [5, 30], a reactive automation's output is a future sequence of environment interactions. Because no external source can pre-label this sequence, the verification reference must be actively constructed on the fly. Second, syntactic divergence complicates evaluation. The same user intent can be expressed through many syntactically different yet behaviorally equivalent implementations. Therefore, checking a generated program by matching it against any single reference text is neither sound nor complete. Determining correctness fundamentally requires reasoning about the program' s semantic behavior over time, rather than analyzing its static syntax or a single snapshot of its output.

Running example: Suppose a user requests "whenever the temperature rises above 25℃, turn on the air conditioner." We read "rises above" as an edge-triggered intent: the AC should be switched on once per upward crossing, not continuously while it stays hot. Figure 1 shows three lowerings, sketched in JoI, the automation DSL of our target platform (§4). Implementation (a) detects the crossing by comparing the previous and current temperature readings while (b) keeps a flag that is set on firing and reset when the temperature falls back below the threshold. (a) and (b) share almost nothing syntactically yet behave identically, so no syntactic comparison can certify one against the other. Implementation (c) appears to be the most intuitive translation because it textually mirrors the natural-language request. However, it is the wrong one: under periodic execution it falsely re-fires on every tick as long as the temperature stays above the threshold. All three are syntactically valid and look plausible; they can only be differentiated by evaluating their execution traces over time; because repeated On() commands leave the AC in the same visible state, the user may see nothing wrong. This is a crucial failure mode our system eliminates: **deployed code that silently diverges from the user's intended behavioral specification**. (In the sketches, `cron` sets when the script starts, `period` is the tick interval in milliseconds, and `code` is the body executed on every tick; `:=` initializes a persistent variable once, while `=` performs per-tick update.)

Why existing checks fall short. Prior verification frameworks of IoT automations target fixed, predefined criteria rather than the user's intent. A large body of work checks hand-authored trigger-action rules [26]; for instance model checkers prove safety properties, inter-rule conflicts, or security policies (e.g., AutoTap [33], IoTSan [17], IoTGuard [3]), while runtime monitors flag

```
cron:""
period: 1000
code:
"prev := 0
current = (#TemperatureSensor).temperature
if (prev <= 25 and current > 25) {
   (#AC).on()
}
prev = current"
```

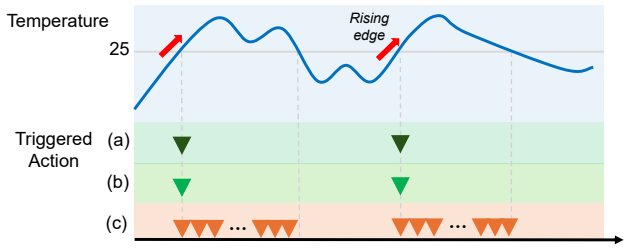
(a) Previous/current comparison

```
cron:""
period: 1000
code:
"triggered := false
temp = (#TemperatureSensor).temperature
if (temp > 25) {
   if (!triggered) {
        (#AC).on()
        triggered = true
   }
} else {
   triggered = false
}"
```

(b) Triggered flag

```
cron:""
period: 1000
code:
"wait until ((#TemperatureSensor).temperature > 25)
(#AC).on()"
```

(c) Re-fires while the level holds



(d) Action traces

Figure 1: Three lowering sketches of one intent ("whenever the temperature rises above 25°C, turn on the air conditioner") and their action traces. (a) and (b) produce identical traces; (c) fires on every tick while the threshold is exceeded.

deviations from mined invariants [10]. This limitation is a consequence of design paradigm, not oversight. When a user writes the rule directly, the rule itself serves as the ground-truth intent, and critical risks worth verifying, such as rule conflicts and security violations, can be defined, without referencing the unique intended behavior of a specific task. However, as the user transitions from author to prompter, the verification paradigm fundamentally changes. Once an LLM synthesizes code from a natural-language request, the user' s original statement and the generated artifact become distinct, decoupled entities that can silently diverge. In this generative workflow intent-conformance emerges as a critical, unaddressed checkpoint.

In that generation setting, the gap remains wide open, and bridging it hinges on a single question: does a test oracle already exist? Generators designed for algorithmic IoT code can readily validate their output against executable test suites or labeled accuracy, similar to how text-to-SQL frameworks verify relational denotations. For these domains, intent-conformance is effectively a solved problem (e.g., GPIoT [23], AutoIOT [24]). Reactive automations, however, lack any such ready-made oracle. Hence natural language-to-automation generators must settle for weaker proxies: schema validity (e.g., verifying that the framework accepts the JSON syntax), device grounding feasibility, lexical matching against ground-truth commands, or evaluation by an LLM judge [35]. Even frameworks that present the generated rule back to the user rely on manual, error-prone human-in-the-loop confirmation rather than formal verification of the underlying executable code (e.g., AwareAuto [25]). To our knowledge, no existing system deterministically checks that reactive-temporal code faithfully executes the user's behavioral intent over time. The fundamental missing piece is not a more capable language model, but a verifiable reference specification.

Why not an LLM judge. The remaining alternative that targets user intent directly is to ask another LLM whether the code matches the request. But an LLM judge is too unstable to act as a deployment gate. A deployment decision is binary: two behaviorally identical programs must receive the same deploy/reject verdict. Yet we find that on behavior-preserving programs that differ only in idiom, an LLM judge's verdict flips even at temperature 0 (§3). Larger models still flip, and cloud models violate the privacy and latency constraints of the edge. A gate whose verdict tracks the surface form of the code is not fit to gate deployment.

OVLA. To address these challenges, we present OVLA. Its key idea is to *derive the missing reference specification* at authoring time and subsequently utilize it as a deterministic verification baseline. First, OVLA builds a candidate Timeline IR from a natural-language request. Then, the system generates a deterministic, plain-language rendering of this IR for the user to review and confirm. This confirmed IR serves as a per-automation behavioral oracle. The generated JoI code is then verified against the oracle by bounded trace-equivalence over the finite set of transition points defined by the IR. Both components are run on events synthesized at those points (e.g., timer expirations, condition thresholds, and cycle restarts), and their resulting action traces are compared. While the LLM is confined to *generation*, proposing the IR and lowering it to code, both the rendering step and the equivalence check are deterministic and LLM-free. This allows conformant automations to be safely deployed at the edge without cloud-side review. *Precisely because* lowering relies on a probabilistic LLM, a deterministic checker is needed.

Scope and assumptions. OVLA targets natural-language commands specifying explicit reactive-temporal behaviors. Inferring intent from vague or affective goals (e.g., "make it cozy") is a separate intent-elicitation challenge addressed by orthogonal literature (e.g., Sasha [15], SAGE [21]) and is strictly out of scope. Any residual ambiguity within a specifiable command is intentionally surfaced by OVLA's deterministic rendering phase and resolved during user confirmation, rather than by the verifier itself. Confirming the IR is presented as a plain-language judgment rather than

a programming task. The dimensions a user controls in reactive-temporal automation are limited and enumerable (start time, period, trigger and condition, action and its arguments, sustain and delay durations), Timeline IR expresses only these slots, and deterministic rendering displays them verbatim as a readable timeline (§8.1). Approving an IR is thus a closed judgment over finite control slots the command named, not the authoring of arbitrary formal properties.

Two-layer guarantee. We explicitly define the boundaries of our verification scope. In **Layer A (NL→IR)**, the system captures intent as IR and presents it in plain language, but we do not measure non-expert confirmation success *itself* in this round via a human-subjects study (§9). In **Layer B (IR→code)**, once the IR is approved, OVLA prevents the generated code from silently diverging from that displayed IR within a bounded behavior region. OVLA's technical guarantee lies firmly in Layer B.

Deployment setting. We assume a home edge device rather than the cloud, to ensure privacy (sensor data never leaves the home) and reduce latency. This environment precludes a cloud LLM judge, requiring the validation gate to be an LLM-free deterministic check. Consequently, a single design decision simultaneously delivers trust (via a non-probabilistic judge) and deployability (millisecond-level localized execution with zero model-call overhead).

Contributions. To the best of our knowledge, OVLA is the first system for LLM-generated reactive-temporal IoT automations that transforms intent-conformance into a deterministic, pre-deployment verification check. It achieves this by establishing a user-confirmed behavioral IR as the authoritative test oracle, rather than relying on a probabilistic judge or superficial manual review.

- **C1. Timeline IR:** We introduce a unified representation that simultaneously serves dual roles: (a) a surface that is deterministically rendered to plain language for user validation, and (b) a machine-checkable behavioral reference for validating reactive-temporal code. While an approved NL sentence can serve only (a) and a formal property only (b), our contribution lies in a singular representation that achieves both. Given that prior works can already generate confirmable temporal IRs, our distinct claim is the novel integration of deterministic, LLM-free verification, automatic rendering, and localized on-device operation (§2).
- **C2. Deterministic deployment gate:** We propose an end-to-end pipeline, spanning IR→FSM→boundary event→bounded trace-equivalence, that executes entirely without LLM intervention. The gate operates under a fail-closed policy where a pass triggers immediate deployment and a failure triggers automatic repair or rejection. When the gate rejects a simulatable program, the rejection is backed by an executable counterexample: a real trace mismatch under the defined observation model (rejection-soundness).
- **C3. On-device realization:** The generation phase utilizes a 4-bit quantized model of at most 9 billion parameters without any fine-tuning, while the verification gate bypasses LLM execution entirely. Consequently, verification completes locally in about a millisecond at the median. OVLA's verification guarantee does not depend on the size or quality of the generation model.
- **C4. Evaluation:** We evaluate rendering faithfulness (§8.1) and verifier detection accuracy (§8.2), empirically demonstrate system safety by mitigating genuine silent IR-to-code divergence of 8.6% (9.2% flagged; strongest generator) down to zero (§8.3), and measure on-device execution overheads and deployment feasibility (§8.4) through rigorous component-by-component analysis.

Focus of the contribution. OVLA removes a failure mode that survives user approval: even when the approved IR is acceptable, the subsequent reactive-temporal lowering phase can introduce subtle state, timer, and boundary errors that cause the deployed code to diverge. OVLA serves as the deterministic, on-device gate specifically engineered to intercept and remediate exactly these errors.

2 Related Work and Positioning

Verification of home automations did not begin as an open problem. When users authored rules directly, the rule itself was the artifact under analysis, and formal techniques could check it against fixed properties; when code was generated in domains that ship executable tests, the tests served as the oracle. In both settings, a checkable reference existed before verification began. LLMs entered this workflow for a practical reason: authoring is the bottleneck, as end users struggle to write and debug even slot-based rules, and richer reactive-temporal behaviors require code they cannot write at all. This shift turns the user from an author into a prompter, separates the request from the artifact that implements it, and removes the very reference that earlier verification relied on. We therefore organize prior work by the reference each system checks against.

When a verification reference already exists. A rich body of work verifies hand-authored TAP rules with formal techniques. AutoTap [33] synthesizes or repairs TAP rules, modeled as transition systems, so that they satisfy LTL safety property templates chosen by the user. TAPInspector [31] model-checks timing-aware rule sets for safety and liveness violations and TAPFixer [32] repairs such violations; related analyses uncover inter-rule and security vulnerabilities [2, 28]; and HAWatcher [10] monitors deployed automations against mined invariants at runtime. Deterministic verification is possible in this line because the reference is fixed in advance: a safety property, conflict freedom, or a security policy. Because the user authors the rule directly, the rule is itself the artifact under analysis, and a separate per-task intent reference is never required.

The same holds for generation targets whose domain supplies an executable oracle. GPIoT [23] generates signal-processing and ML algorithm code from natural-language requirements using an on-device fine-tuned SLM and verifies the result with pass@k execution tests. Outside IoT, text-to-SQL verifies a generated query against the denotation of a gold query [30], and CodeT [4] and Self-Debug [6] verify generated code with unit tests. *Across both lines, verification was enabled by a checkable target that was already available, whether a fixed property, a test suite, or a gold denotation.*

LLM-generated reactive automations: no ready-made reference. LLM-generated reactive automations have no such target:

the intended behavior is named only in the user's request, and no benchmark or test suite labels its future action sequence. The missing reference is also harder to substitute here than for TAP rules. TAP rules do carry temporal behavior, including triggers, stays-for durations, and cron-style schedules, but there it is a platform *primitive*: the author fills slots and the engine implements the mechanism, so slot errors are visible on the face of the rule. In reactive-temporal code such as JoI scripts, no such primitive exists for edge detection, sustained conditions, or counting; the author, here an LLM, must realize the mechanism as an *idiom* built from persistent variables, conditions, and per-tick execution. Mechanism errors are silent, and they are exactly where lowering bugs arise.

Some systems in this regime do check LLM output deterministically, but still against fixed criteria. AutoIoT [7] checks LLM-generated rules against four inter-rule conflict types using Maude rewriting logic. DS-IA [14] checks the device-grounding feasibility of a command with a deterministic cascade and rejects infeasible commands fail-closed before execution. TaskSense [16] translates a sensing query into an executable tool-calling DAG, screens it with an LLM-based solvability check, and then deterministically checks that the plan's dependency graph is a subgraph of the toolset grammar; both checks establish structural well-formedness, and its only behavioral reference, a hand-authored ground-truth plan, exists solely in offline evaluation. AgentSpec [27] enforces trigger-predicate safety rules on LLM agents at runtime through a deterministic DSL; however, its rules are fixed and hand-written rather than derived from the request being served. *Most of these checks are deterministic, yet their reference remains a predefined criterion; the behavior the user actually asked for is still never the reference.*

Generation systems themselves fall back to weaker checks. ChatIoT [11] translates natural language into Home Assistant automations with multiple LLM agents and has a separate "Evaluator" agent assess format compliance and request satisfaction; Giudici et al. [13] accept a generated routine if the framework parses its JSON; and IoTGPT [29] catches API violations and runtime errors, though not intent mismatches, in a virtual SmartThings environment. AwareAuto [25] goes furthest: it builds a confirmable reactive-temporal rule with event/state modes, delays, branches, and sequencing, presents it to the user in a panel, and lowers it to executable JSON. *Across this line, automatic checking stops at format adequacy, API validity, or device-grounding feasibility. Even in AwareAuto, intent consistency is evaluated through researcher annotation, and the user-facing confirmation text is generated prose rather than a deterministic rendering. Consequently, none of these systems verifies that the generated code behaves as the user intended.*

Work that directly asks whether the generated artifact matches the user's intent remains rare. LACE [8] back-translates a generated access-control policy into natural language with deterministic templates and judges semantic equivalence against the original request with a fine-tuned NLI model. SimuHome [22] judges agent behavior against goal states in a time-accelerated simulator and shows that 18 LLM agents largely fail to self-correct their own reactive-temporal errors (§3); its goals, however, are authored by the benchmark rather than confirmed by a user, and it evaluates agents rather than gating deployment. *Even where intent is addressed directly, intent has no ground truth other than the user, so a model-judged verdict certifies itself, as in LACE's reported "100% correctness".*

Two systems come nearest to OVLA, each along one axis of our problem: LACE on the reference axis, since it checks a generated artifact against the user's request, and AwareAuto on the artifact axis, since it builds a confirmable reactive-temporal representation. LACE, however, judges probabilistically over static policies, and AwareAuto never verifies the behavior of the lowered code. OVLA starts precisely where both stop, treating the user as the only legitimate source of the reference: it derives the missing reference at authoring time, takes the user-confirmed IR as the behavioral oracle, and checks the generated reactive-temporal code through deterministic, LLM-free trace-equivalence, before deployment and on the device. We note that merely combining generation with verification is standard practice in this field and is not our claimed novelty; the novelty is that the user-confirmed IR itself becomes the executable behavioral oracle, which is what makes deterministic, LLM-free, on-device checking of reactive-temporal code possible; no single prior system provides this.

3 Problem and Motivation

Together, §1 and §2 leave a clear task: a gate is needed that decides, before deployment, whether LLM-generated reactive-temporal code behaves as the user intended. This section first establishes the requirements such a gate must meet in our setting. Next it weighs alternative candidate checking approaches against them, and finally measures the one remaining candidate, an LLM judge, motivating our proposed solution.

Three requirements for a deployment gate.

- **R1 (Reference).** No prior reference exists. The intended behavior is determined solely by the user' s recent request, meaning it is not provided as a checkable artifact. Algorithmic code carries an executable oracle in the form of input-output pairs and is checked by running tests (GPIoT [23], §2). For reactive-temporal code, the "right output" is a future sequence of device actions that no external source labels. The reference must be constructed, not looked up.
- **R2 (Stability).** A deployment decision is binary: two programs with the same behavior must receive the same deploy/reject verdict. A checker whose verdict wavers under behavior-preserving rewrites is unsuitable as a gate, regardless of the quality of its individual judgments.
- **R3 (On-device).** Sensor data must not leave the home (privacy) and the verdict must be immediate (latency), so the gate must run on the edge without a cloud round-trip.

Candidate 1: syntactic or structural comparison. The most straightforward verification method is to match the generated code directly against a reference text. However, as demonstrated by the temperature control example in §1 (Figure 1), identical user intent can be realized through syntactically disparate idioms (a prev/curr comparison vs. a triggered flag), while the buggy implementation is the one that most resembles the request. Indeed, near-miss bugs differing by only a single execution tick or a single comparison can appear identical to valid code variants. Because syntactic or structural matching can neither validate diverse correct variants

nor isolate subtle bugs, it satisfies neither soundness nor completeness. Equivalence must be decided on execution behavior, not on syntax.

Candidate 2: self-checking by the generating model. One could ask the model that produced the code to inspect and repair it. SimuHome [22] evaluated this method with 18 models; the self-correction recovery rate was 8.0% for time-based errors, 18.5% for event-based errors, and 0.0% for coordinated scheduling. Notably, the success rate remained at or below 67% even when an oracle was provided. Based on these findings, the authors concluded that agents fail to detect flaws in their own plans. A post-hoc self-check entrusted to the same stochastic process that generated the code cannot serve as a reliable quality gate.

Candidate 3: human inspection of the code. One could show the generated code to the user for approval. However, prior research in TAP-Debug [34] demonstrated that non-experts frequently misread even the basic IF (event) and WHILE (state) conditions of raw rules; specifically, misreadings occurred in 21 out of 50 control-condition sessions, all of which subsequently failed the task. Given that such errors occur even with a simple, two-slot TAP rule, it is untenable to expect users to successfully inspect complex reactive-temporal code, which realizes its mechanism through persistent variables and per-tick execution. What is shown to the user must be a readable representation rather than the raw code; this also motivates the deterministic rendering detailed in §5.

The remaining candidate: an LLM judge. What remains is to hand the request and the code to a separate LLM to verify whether they align. This approach appears plausible at first glance and is indeed used in practice (ChatIoT's Evaluator, §2), and existing literature does not rule it out. We therefore evaluate whether this candidate meets R2 independent of OVLA's IR and verifier.

We presented pairs of programs exhibiting identical behavior (trace-exact) but differing in idiom, and measured how often the judge's accept/reject verdict flipped. Even under a deterministic setting at temperature 0, a 9B model flipped on 27% of pairs and a strong cloud-based model (GPT-5.1 [18]) flipped on 10.6%. The rewrite types are categorized into surface rewrites (SEL = selector order, AND = $A \wedge B \leftrightarrow B \wedge A$, VAR = variable renaming) and logic/arithmetic rewrites (ADD = $x+n \leftrightarrow n+x$, BR = branch swap, DN = double negation, DM = De Morgan, IDM = idiom replacement). As shown in Figure 2, purely surface rewrites flipped up to 9% of the 9B judge's verdicts (14% for the cloud judge), while logic and arithmetic rewrites caused flips up to 81%. No ground-truth label is required to demonstrate this; the flip itself evidences instability.

The core is not the miss rate; on cleanly injected bugs, strong models are indeed competent detectors. The point is that *any* probabilistic judge yields an unstable deployment policy under behavior-preserving rewrites. This instability persists in larger models, and majority voting fails to reduce it below the temperature-0 baseline, because voting mitigates sampling noise but cannot counteract the systematic dependence on surface form (Table 1). This violates R2, disqualifying the final candidate. In contrast, OVLA's verdict depends only on behavior over a bounded set of traces, so under the same rewrites its flip rate is 0 by construction.

The surviving design. With every candidate eliminated, rereading the requirements reveals the shape of the answer. By R1, the reference must be *derived* from intent the user has confirmed; by R2, the check must be deterministic over behavior (traces), not syntax; by R3, it must be lightweight on the edge, with no LLM call. The artifact that satisfies all three at once is the Timeline IR (§5), and the deterministic trace-equivalence check over it is the verifier (§6). The judge in this motivating experiment compares NL against code.

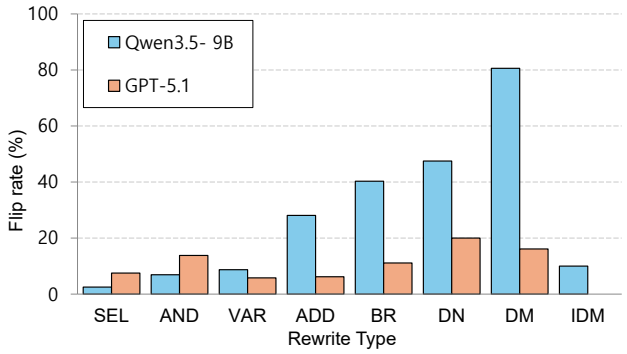


Figure 2: LLM judge verdict flip rate by rewrite type on behavior-identical programs (temperature 0).

Table 1: Overall flip rate by judge configuration; OVLA's trace check flips on none of the same rewrites.

Judge configuration	9B	GPT-5.1
deterministic (temp=0)	27.0%	10.6%
+ sampling (temp=0.7)	34.4%	12.8%
+ majority vote (K=5)	31.2%	13.5%

4 System Overview

OVLA implements the aforementioned design as a two-phase pipeline. In the authoring phase, the system derives a Timeline IR from the user's command, renders that IR for confirmation, and lowers the confirmed IR to executable code. In the verification phase, the system checks the generated code against the confirmed IR before deployment. The central boundary is fixed throughout the pipeline: the LLM may propose candidate artifacts, but every artifact that can affect deployment is checked by deterministic code.

Execution target. We instantiate OVLA on the edge hub of a commercial IoT platform whose execution DSL is JoI. A JoI automation consists of a start schedule (`cron`), a re-execution interval (`period`), and a script body (`code`). When the `period` parameter is positive, the body is executed on each tick, and temporal behavior such as edge detection, sustained conditions, and counting must be implemented with persistent variables and ordinary conditionals. JoI therefore serves as the executable artifact produced by the lowering phase as well as the primary artifact checked by the verifier.

Figure 3 shows the complete pipeline. The first phase, authoring, includes LLM-based generation steps and deterministic gates that execute before the user approves the reference. The second phase, verification, takes the approved IR and the generated JoI code as inputs and runs without any LLM call.

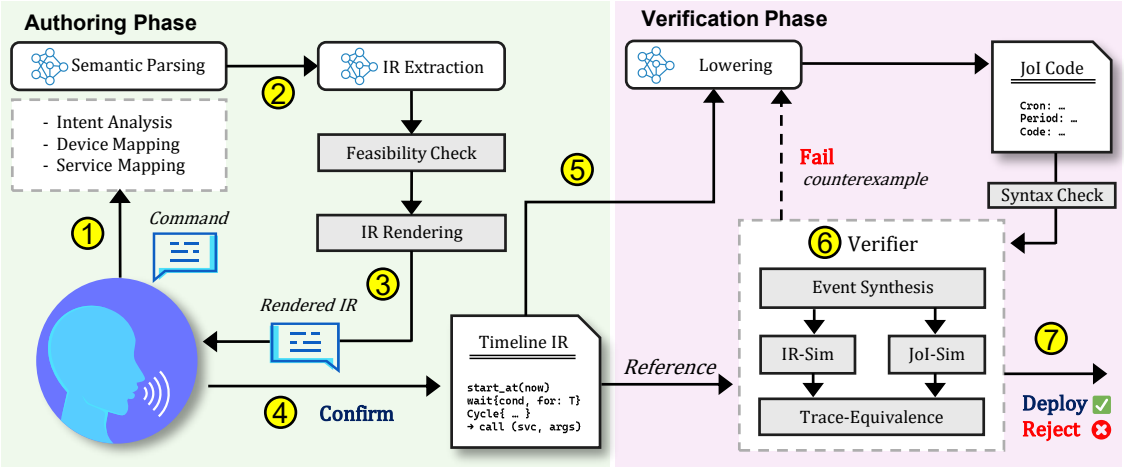


Figure 3: OVLA system overview. (1)–(5) Generation phase and (6)–(7) verification phase; the dashed arrow is the counterexample-guided repair loop.

Phase 1: Authoring (LLM generation with deterministic gates). (1) Command Analysis: OVLA first analyzes the user's natural-language command. This analysis identifies the temporal and behavioral slots required for the automation, such as the start time, repetition period, trigger or condition, delay or sustain duration, and target actions. It also maps the command onto the connected device list, selecting the relevant devices and the services that those devices must expose for the automation to be executable. (2) Candidate IR Extraction: OVLA then extracts a candidate **Timeline IR** from this analysis. The word "candidate" underscores that, prior to user confirmation, the IR merely represents the system's proposed interpretation of the command. Before code generation begins, a deterministic **feasibility gate** (§5) checks whether this IR complies with supported structural rules. It rejects structures that cannot be realized by the target execution model, such as nested cycles or misplaced branches. (3) IR rendering: If the IR passes this gate, OVLA renders it into the deterministic **Rendered IR** shown in Figure 3. The Rendered IR is a plain-language view of the IR, not a new natural-language command. (4) User Confirmation: The user reviews this Rendered IR. Once confirmed, the Timeline IR is finalized as the reference specification for the automation. (5) Lowering to Code: Only after this reference is fixed does OVLA lower the IR to JoI code.

Phase 2: Verification (deterministic, LLM-free). (6) Verification Execution: The verifier receives two artifacts: the confirmed Timeline IR, which serves as the behavioral reference, and the generated JoI code. The verifier first performs a static syntax check on the JoI code. The behavioral check then follows the lower half of Figure 3. From the confirmed IR, OVLA derives an FSM that makes the enabled obligations and their reachable states explicit. The **Event Synthesis** stage generates boundary events from this FSM (§6). The same synthesized event set is fed into two deterministic simulators: **IR-Sim**, which executes the confirmed IR, and **JoI-Sim**, which executes the generated JoI code. Their action traces are passed to the **Trace-Equivalence** stage, which decides whether the code matches the approved IR under the observation model of §6. (7) Deployment and Feedback Loop: A passing program is deployed. A failing program is not deployed directly. Instead, the verifier returns a **counterexample** to the lowering stage to initiate regeneration of the JoI code alone. The confirmed IR is not changed during this repair process; it remains the user-approved reference. If the repair budget is exhausted, OVLA rejects the automation under a fail-closed policy.

Three checks, three places. OVLA uses three deterministic checks, each operating on a different artifact. First, the **feasibility gate** checks the structure of the *IR* as a typed tree, immediately after IR extraction and before lowering (§5). Second, the **static syntax check** validates the *generated JoI code*, after lowering and before simulation. Third, the **behavioral trace-equivalence check** compares the behavior of the confirmed IR and the generated JoI code (§6). These checks serve independent, non-interchangeable purposes: the behavioral check is the core deployment gate, while the feasibility and syntax checks prevent malformed artifacts from reaching that gate.

Roles of the IR. A single Timeline IR plays four roles in OVLA. First, it is the artifact exposed to the user through deterministic rendering. Second, upon user confirmation, it becomes the behavioral reference used by the verifier. Third, it serves as the structural scaffold from which the system lowers JoI code. Fourth, its structural signature is used to route lowering exemplars (§7). While OVLA relies on semantic analysis and device/service mapping in its upstream stages, the primary contribution of this paper centers on how the confirmed IR is rendered, fixed as a reference, lowered, and verified.

5 Timeline IR

Timeline IR serves as the critical artifact connecting user confirmation to deterministic verification. While the NL→IR translation relies on an LLM, the IR→rendering process does not. Because the latter is a deterministic transformation, the text presented to the

Timeline IR

```
{"timeline": [
  {"op": "start_at", "anchor": "now"},
  {"op": "cycle", "until": null, "period": "100 MSEC",  "body":[
    {"op": "wait",
    "cond": "TemperatureSensor.Temperature > 25",
    "edge": "rising"},
    {"op": "call",
    "target": "AirConditioner.On",
     "args": {}}
  ]}
]}
```

Rendering Result

Start:
- Start now.

Repeat:
- Watch for each new occurrence where the temperature of the temperature sensor is above 25.
- Run the following behavior once for every new occurrence.

Behavior:
- Turn on the air conditioner

(a) Recurring edge trigger

Timeline IR

```
{"timeline": [
  {"op": "start_at", "anchor": "cron", "cron": "0 21 * * *"},
  {"op": "cycle", "until": null, "period": "10 MIN",  "body":[
    {"op": "if",
    "cond": "HumiditySensor.Humidity > 70",
    "then": [
      {"op": "call", "target": "Switch.On", "args": {}},
      {"op": "delay", "duration": "5 MIN"},
      {"op": "call", "target": "Switch.Off", "args": {}}
    ]}
  ]}
]}
```

Rendering Result

Start:
- Start at 9:00 PM every day.

Repeat:
- Every 10 minutes, check the humidity.

Behavior:
- If the humidity of the humidity sensor is above 70, turn on the switch for 5 minutes.

(b) Scheduled periodic check with branch and delay

Figure 4: Two Timeline IRs and their deterministic plain-language renderings.

user is a faithful representation of the underlying IR that will subsequently be verified. The rendering cannot introduce behavior absent from the IR, nor omit behaviors contained within it. In this sense, what is displayed is what is checked. The Timeline IR grammar is summarized in the appendix.

Closed control dimensions and primitive ops. The control dimensions of a reactive-temporal automation are limited and enumerable. A user command may specify the following dimensions: when the automation starts (`start_at`), how often it repeats (`cycle.period`), what trigger or condition it waits for (`wait`'s condition and edge, or `if`'s condition), what action to perform and with what arguments (`call`), how long to wait or delay (`wait.for`, `delay`), and how to count or stop repeated behavior (`cycle.count`, `break`). Timeline IR maps primitive operations to these distinct dimensions and places the named slots directly on a timeline.

This representation is intentionally minimal. It avoids requiring the user to approve arbitrary code or write formal properties. Instead, the deterministic rendering exposes a bounded set of control slots specified by the command, ordered precisely as the behavior unfolds. User approval is therefore a closed judgment over those explicit slots: whether the displayed start time, period, trigger, condition, action, duration, and repetition behavior match the intended automation. The same slots are also the ones the verifier consumes. Thus, the IR serves a dual purpose: as an intuitive approval surface through its rendering, and as a rigorous behavioral reference for validating the generated code. The structural syntax G defined below ranges exactly over this closed set of operations.

Finite-state view. Timeline IR models an automation as a finite set of obligations on a timeline. An obligation can manifest as a scheduled check, a wait whose condition must become true, a branch whose guard selects a path, a call that must be issued, or a cycle body that must repeat at the next period. The IR admits no recursion and no unbounded loops. It uses bounded timers, counters, branches, and cycles, so the verifier can enumerate the relevant obligations within its checking horizon. Timeline IR is not intended to be a full timed-automaton model-checking language [1]. It provides a highly operational view that renders the target behaviors both finite and explicit.

Typed tree and structural syntax G. Structurally, an IR is a typed tree. A structural syntax G specifies which parent-child relationships and control structures are legal (the full grammar is given in Appendix A). For example, an IR has exactly one `start_at` and at most one top-level cron schedule. A `cycle` cannot appear inside another `cycle.body`. The `then` and `else` blocks can appear only as children of an `if`. A `break` is legal only inside a `cycle`. A `cycle` must have a period; its `until` condition is optional, and if it is absent the cycle repeats within the verifier's bounded checking horizon (§6).

Whether an IR satisfies these structural rules is deterministic to decide. The same pass over the typed tree also produces the IR's structural class, $\tau(\text{IR})$, which records the combination of rules used by that IR. OVLA uses $\tau(\text{IR})$ later to route lowering exemplars with matching structure (§7). This structural class helps generation, but it does not affect the verifier's accept/reject decision.

Feasibility gate. The feasibility gate in §4 checks the structural validity of the extracted IR against G. It runs immediately after IR extraction and before lowering. If the extracted IR contains a nested cycle, a second top-level cron, a misplaced `then`/`else`, or a `break` outside a cycle, the gate rejects it before any JoI code is generated. This decision is a structural decision, not an empirical result: the gate checks the IR against the rules of G by construction.

This gate does not prove that the IR captures the user's true intent. It only establishes that the IR is structurally supported by OVLA's execution and verification pipeline. If the LLM extractor

silently changes an unsupported request into a different but legal IR, the feasibility gate will accept the legal IR with the wrong meaning. That residual semantic mismatch is intentionally left to user confirmation through rendering.

The primitive operators comprise `start_at`, `wait` (condition, edge, and sustain duration), `delay`, `read`, `call`, `if` (then/else), `cycle` (period, count, until, and body), and `break`. Figure 4 illustrates this design with two Timeline IR examples and their deterministic renderings. Each example shows how internal IR slots map directly onto user-facing approval text.

Figure 4(a) revisits the temperature example from §1. The phrase "whenever the temperature rises above 25℃" is formalized into a `wait` operator whose condition is `Temperature > 25` and whose edge is rising, placed inside a `cycle`. The rendering exposes this edge slot to the user as "each new occurrence" and the `call` operator as "Turn on the air conditioner." Figure 4(b) shows a schedule-driven automation. The `start_at` cron is rendered as "Start at 9:00 PM every day," the `cycle.period` slot is rendered as "Every 10 minutes," and the `if-call-delay-call` sequence is rendered as one conditional behavior. This rendering faithfulness is evaluated in §8.1.

6 Verifier

6.1 Observation Model: Transition-Boundary Conformance

The gate decides a single question: does the generated JoI code *exhibit the same behavior* as the approved IR? As observed in §2, the execution DSL has no primitives for the IR's control dimensions, so the LLM realizes each IR behavior as an idiom. For instance, "sustained for N minutes" is implemented as a counter that increments on each poll tick and fires upon reaching a threshold. Similarly, "edge (whenever)" becomes a triggered flag set on firing and reset when the condition clears (or a prev/curr comparison), while "every N minutes" becomes periodic re-execution of the body. One IR behavior thus appears as several valid idioms and as countless near-misses one tick or one comparison off (Figure 1). This idiom-realization point is exactly where bugs enter. Equivalence must therefore be decided based on execution behavior (traces), not on syntax.

OVLA does not attempt to prove equivalence over every possible environment trace. Instead, it checks the points at which reactive-temporal behavior can change. These transition points include guard thresholds, condition edges, time landmarks such as delays and periods, and internal state transitions. Internal state transitions are changes in the program's own persistent state, such as a triggered flag being set or reset, a sustain counter reaching its firing threshold, or a cycle counter reaching the value that causes a break. They must be included because the same sensor value can lead to different behavior depending on the internal state: for example, the flag-based implementation in Figure 1(b) may fire before the flag is set and stay silent afterward.

The checked space is finite for two reasons. First, Timeline IR admits only bounded control: no recursion and no unbounded loops, so the reachable internal states within a run are finite. Second, the verifier uses a bounded time horizon. In our implementation, the verifier simulates one week, which contains a full recurrence of every time-of-day and day-of-week schedule. Calendar fields such as day-of-month and month do not require longer simulation. They only select on which dates the weekly pattern is active; they do not change the behavior inside that active window. The verifier therefore statically checks that the IR and the JoI code name the same calendar dates, and then reuses the already-simulated weekly behavior.

We call this observation model **transition-boundary conformance**. Under this model, two programs conform if their action traces agree, within the verifier's tolerance, at the boundaries where behavior can change: threshold crossings, condition edges, timer expirations, and next-cycle restarts, over the fixed bounded horizon. This is a boundary-focused equivalence judgment, not a proof over the entire input space. The restriction is deliberate. It makes the check deterministic, solver-free, and small enough to run on the edge hub. The verifier should therefore be read as a deployment-time filter, not as a replacement for heavyweight formal verification [9].

6.2 Boundary Event Synthesis

The event synthesizer is deterministic and LLM-free. It starts from the approved IR and derives an FSM that records which IR operations are active at each point in the execution. Each primitive operation contributes an obligation: `call` contributes an action obligation, `wait` and `delay` contribute temporal obligations, `read` contributes a value-observation point, `if` contributes guarded branches, and `cycle` and `break` contribute iteration and termination structure. Sequencing between operations imposes order, branches add guards, and cycles add the obligation to repeat the body at the next period.

The synthesizer's goal is to create concrete events for the boundaries identified by the observation model. For a rising-edge condition such as `wait(Temperature > 25, edge: rising)`, a boundary is the moment when the temperature changes from at most 25℃ to above 25℃. For a `delay`, the boundary is the time when the delay expires. For a `cycle`, the boundary is the start of the next period.

These boundary events cannot be placed arbitrarily. The same temperature crossing matters only if the automation has already reached the `wait(Temperature > 25, edge: rising)` operation. It is not useful before earlier waits or delays have completed, on a branch that was not selected, or in a cycle iteration that has not been reached.

The synthesizer therefore needs to know where the automation is in its execution. It must know which waits and delays have completed, which branch was selected, which cycle iteration is active, and which IR operation is currently enabled. The FSM records this execution state explicitly.

Using this FSM state, the synthesizer places each representative event at a time when the corresponding IR operation is active and able to change the trace. Within a cell, the exact value crossed is not the important part; the timing and reachability of the event are. When a mismatch is found, the verifier attributes it to the obligation associated with that boundary, and that attribution becomes the counterexample returned to the repair loop of §4.

The synthesis is based on IR semantics rather than JoI surface syntax. It tracks read→sensor flows and seeds both ends of each relevant value domain, so scale and offset mistakes can be exposed. For expressions that read the same sensor twice and compare the results, it additionally seeds the relation between the two reads in three directions (equal, rising, and falling), so sign and direction mistakes are exercised. Some IR arithmetic helpers, such as `abs`, `min`, and `max`, have no direct JoI counterpart. Lowering therefore expands them into conditional code, and the synthesizer seeds the internal boundaries of those expansions as well. This prevents, for example, an inverted clamp from passing simply because only ordinary threshold values were tested. The synthesizer also includes second-edge scenarios, where a condition clears and later becomes true again, because those cases are necessary to test whether the trigger can fire again after the condition clears. Read-modify-write accumulators are excluded from this targeted boundary seeding and are treated as a residual limit of the check. The final synthesized event set is fed unchanged to both the IR simulator and the JoI simulator, so the two artifacts are compared on identical inputs.

6.3 Trace-Equivalence and Omission Detection

The two simulators run on the same synthesized events and produce action traces. The verifier groups and deduplicates actions within a timing tolerance that absorbs tick quantization and the lowering's choice of polling period. Subsequently, it compares the traces sequentially, preserving behavior-relevant differences in action order, multiplicity, and boundary timing while avoiding verdict changes caused only by harmless polling-level timing variation.

This trace comparison detects both commission and omission. A commission error occurs when the JoI code performs an action that the IR does not allow. An omission error occurs when the IR requires an action and the JoI code fails to produce it. Significantly, silence is part of the trace. If the IR trace contains an empty interval, that interval is an oracle saying that no action should occur there.

This is why OVLA compares the JoI trace against the expected action trace produced by simulating the approved IR. The IR simulator does not only produce the actions that should happen; by producing no action during a time interval, it also specifies that the interval should remain silent. For example, if the approved IR says "when the door opens, turn on the light after 5 minutes," the IR trace contains no action during the five-minute delay and then contains `Light.On()` at the end of the delay. Code that turns on the light immediately may reach the same final device state, but its JoI trace contains an action inside an interval where the IR trace is silent. Trace-equivalence detects this timing mismatch.

6.4 What the Verdict Guarantees

The verifier yields a deterministic, LLM-independent verdict. It provides two properties:

- **Determinism(T1)**: the same approved IR and generated JoI code always produce the same verifier verdict.
- **Rejection-Soundness(T2)**: when the verifier rejects a program, it has found a real trace mismatch under the observation model defined above. In other words, rejection is tied to an executable counterexample, not to a probabilistic judgment.

We do not claim absolute correctness upon passing ("pass ⇒ correct"). A pass means only that the generated code matches the approved IR on the synthesized boundary traces within the bounded horizon and tolerance. The trusted computing base is strictly confined to the deterministic verifier: the IR simulator, the JoI simulator, the tolerance policy, and the trace-equivalence relation. The verdict therefore does not rely on trusting the LLM that generated the code. The soundness of a rejection reduces to simulator fidelity rather than to a proof over all environments. We evaluate the check empirically in two ways: §8.2 stress-tests verifier detection and the adequacy of the synthesized traces with construct-derived mutation tests and coverage measured on both the IR-FSM obligations and the generated JoI branches, and §8.3 measures the deployment outcome of applying the gate to generated automations. Surviving mutants are characterized individually.

6.5 Limits of the Check

The limitations of the check stem directly from the observation model. First, the one-week horizon, together with the static calendar check, covers time-of-day and day-of-week schedule behavior. It does not exercise behaviors whose own duration exceeds the horizon, such as a month-long sustained condition, to completion. Such automations can still be represented, but their long-duration behavior is outside the exercised region.

Second, the check is limited by arithmetic lacking explicit boundaries. The three-direction relational seeding of §6.2 exercises sign and direction errors, but an expression whose value flows into an argument, such as `notify($t1-$t2)`, defines no threshold, edge, or timer at which the synthesizer can target a representative boundary. Consequently, value-specific bugs dependent on precise numerical relations may survive, even though generic seeds can still detect sign errors or structural flaws. These limits do not make the two simulators disagree about the same input; rather, they identify places where the synthesized event set may fail to exercise a bug.

7 Implementation

OVLA's deterministic core is pure Python with no external dependencies. The two simulators (about 3.7K lines), the verifier (1.1K), the rendering (0.7K), and the feasibility gate (0.1K) total roughly 5.7K lines of deterministic code, with no solver and no model call. Generation runs on off-the-shelf 4-bit-quantized models of at most 9B parameters, without fine-tuning (§8 Setup). Generation is decomposed into the stages of §4 (intent analysis, device and tool mapping, IR extraction, lowering), and the verifier's guarantee is independent of which generator produces the code.

Structure-based example routing. Instead of carrying few-shot examples for every idiom family, the lowering prompt includes only the examples for the IR's structural class from the feasibility pass (§5), reducing the prompt by 33% on average (38% for acyclic IRs, 19% for cyclic ones) and thus memory and context pressure on the local model. Verified (IR, JoI) pairs, including repaired ones, can be stored under their structural signature as examples for future requests of the same class. Routing affects only the generator's prompt; the accept/reject verdict remains the deterministic

check of §6, and we claim no accuracy benefit from routing in this submission.

8 Evaluation

Our evaluation asks four questions, ordered as the pipeline reaches the user. Does the rendering the user approves faithfully display the IR (§8.1)? Taking the approved IR as the reference, does the verifier catch wrong code (§8.2)? Does the full pipeline, with the verifier as its gate, classify generated code correctly (§8.3)? And is all of this affordable on an edge device (§8.4)? Each answer is a premise for the next: every later check takes the user-approved IR as its reference, and the interpretation of the gate's outcomes rests on the verifier's detection quality.

Setup. *Hardware and models*: The primary edge device is a Mac mini M4 (16GB). Experiments that require many model generations run on GPU servers, while all on-device cost measurements run on the M4. Generation uses Qwen3.5-9B [20], with Qwen3-8B [19] and Gemma-4-E4B [12] as additional generators (Table 5); all are 4-bit quantized without fine-tuning. The deterministic verifier runs identically across backends.

Dataset: The benchmark comprises 382 natural-language automation commands across 24 device/service categories. Commands cover representative reactive idiom families prone to lowering errors in JoI (e.g., triggers, rising and falling edges, sustained conditions, periodic checks, delayed sequences, branches, and bounded counting). The dataset was not curated to fit OVLA's IR schema. Each command is paired with a manually authored reference Timeline IR, independently audited at the value level: start times, periods, trigger edges, sustain and delay durations, branch conditions, target services, and action arguments were checked against the source command. The category breakdown is reported in the appendix B.

8.1 RQ1: Rendering Faithfulness

In OVLA, users approve an automation by reviewing the plain-language rendering of its Timeline IR, which expresses behavioral logic through a finite set of control slots. To ensure safety, any behavioral variance within these slots must reflect as a distinct textual change. If different slot configurations rendered identically, users could not distinguish a faulty IR from a correct one, compromising the verification gate.

Design. We evaluated this fidelity in two parts (Table 2).

- Part A (Synthetic Check): We injected eight fault classes across every applicable slot of the 382 reference IRs, generating 1,504 (correct, faulty) IR pairs. These classes spanned condition slots (comparator flips, polarity inversions, AND-conjunct drops), action slots (argument values, target devices), time slots (durations), and structure slots (oneshot↔wait-until and single↔cycle). A deterministic check flagged a fault as "surfaced" if the rendered texts differed, and "blind" if they were identical.
- Part B (Real-fault Check): To align with realistic LLM error distributions, we evaluated 55 logic-faulty IRs generated during NL→IR extraction by Qwen3.5-9B that successfully bypassed the static validation gates.

Results. Both parts achieved 100% detection with zero blind spots. All 1,504 synthetic pairs and all 55 real logic faults surfaced as distinct text. For example, in the oneshot↔wait-until class the correct IR renders as "If the charging state of the charger is fully-Charged, turn off the selected switch." while the faulty IR renders as "Wait until the charging state of the charger is fullyCharged is true. Turn off the selected switch."

Table 2: Evaluation of rendering faithfulness against synthetic and real logic faults.

Fault class	count	Surfaced (%)
comparator	267	100
polarity	85	100
wrong arg value	235	100
wrong device	382	100
timing/duration	74	100
oneshot↔wait-until	150	100
single↔cycle (whenever)	273	100
AND-conjunct drop	38	100
Synthetic total (Part A)	1,504	100
Real logic faults (Part B)	55	100

8.2 RQ2: Verifier Detection

To evaluate whether the verifier effectively detects incorrect code, we isolate it from the deployment pipeline and stress-test it using synthetic perturbations of correct JoI code. This keeps the detector' s baseline capability separate from the downstream deployment-gate safety evaluation.

Mutation Testing: We take 328 valid (IR, correct code) pairs that pass the verifier as our baseline seeds. From each seed we mutate exactly one spot of the correct *code* using twelve operators (Table 3), including numeric argument changes, enum swaps, call deletions and insertions, tick scaling, guard polarity flips, comparator swaps. We then feed each mutant, together with the original IR, to the verifier. A rejection means the fault is caught; a pass means it is missed (a survivor).

To ensure deterministic coverage, we generate exactly one first-order mutant per applicable site (e.g., 436 call sites yield 436 `call_drop` mutants). This systematic process generated 1,651 mutants (Gen.) as shown in Table 3. We exclude 99 equivalent mutants (Equiv.) whose traces match the original under every scenario (e.g., changes on unreachable branches). Of the remaining 1,552 genuine mutants (Genuine), the verifier catches 1,541, or 99.3% (Caught).

Only three operators in Table 3 fall short of 100%, and none of the 11 survivors represent new structural weaknesses. Eight (`comparator`) replace >= with > on a sustain counter. This merely delayed the action by one tick (100ms), which fell within the timing tolerance designed to absorb polling slack. Thus, passing was the intended behavior. One of the two `call_drop` survivors deletes one of two identical commands sent to two rooms at the same instant. Because the final device state remains identical whether the command is sent once or twice, the deduplicated trace showed no distinguishable difference. The remaining two, the other `call_drop` and the single `cmp_direction` survivor, sit in a single read-modify-write arithmetic automation. As disclosed in §6.2 this residual class is excluded from boundary seeding, making this escape expected.

Table 3: Construct-derived mutation results (328 seeds, 12 operators).

Operator	Gen.	Equiv.	Genuine	Caught (%)
arg_numeric	142	1	141	141 (100)
enum_flip	111	2	109	109 (100)
call_drop	436	4	432	430 (99.5)
call_add	200	3	197	197 (100)
tick_scale	55	0	55	55 (100)
guard_polarity	198	0	198	198 (100)
comparator	159	34	125	117 (93.6)
assign_init	71	13	58	58 (100)
cmp_direction	159	30	129	128 (99.2)
arith_op	37	7	30	30 (100)
break_drop	29	5	24	24 (100)
wait_drop	54	0	54	54 (100)
Total	1,651	99	1,552	1,541 (99.3)

Table 4: Spec-side coverage of the synthesized scenarios: the fraction of IR-defined check points the scenarios actually reach.

Obligation	Total	Covered	(%)
`if`: then branch	166	166	100
`if`: else branch	37	28	75.7
`wait` (edge/sustain/second edge)	122	122	100
arithmetic expr boundary (lo/hi/fall)	34	34	100
Total	359	350	97.5

Coverage. Since detection relies entirely on the synthesized scenarios from §6.2, we evaluated whether these scenarios adequately reach every check point (Table 4). Check points include both sides of every `if` branch, the edge/sustain of every `wait`, and the value boundaries of every arithmetic expression. `call`, `delay`, and `cycle` execute unconditionally and are verified via trace comparison, so they are excluded here. The scenarios reach 97.5% (350/359) of these points. All 9 unreached points are the else side of modulo guards over a cycle's iteration counter (`n % k == c`). Since the counter is not a sensor, no seeded value can steer it. Measuring the same scenarios on the generated JoI code, 97.7% (676/692) of the code's own branches actually execute, proving that the scenarios thoroughly test the implementation, not just the specification.

The 99.3% recall rate warrants a caution: both the mutation operators and the verifier's check points are derived from the same IR constructs. This high score indicates that the verifier effectively catches the specific faults it was designed to target; it does not vouch for bugs outside this defined scope.

8.3 RQ3: System Safety

Finally, we evaluated the full pipeline across all 382 commands, using the verifier as a deployment gate. The gate verifies each lowering candidate, deploys it if it passes, and otherwise enters a repair loop that returns the counterexample to the generator (at most two attempts), rejecting fail-closed if repair also fails. Using the 9B generator, the pipeline yielded the following outcome distribution:

Table 5: Gate classification across generation models (382 automations, confirmed IR). Flagged candidates comprise behavioral divergences and unparseable code rejected before simulation.

Generator (4-bit, no FT)	Flagged	Rep.	Rej.	Deployed
Qwen3.5-9B	35 (9.2%)	11	24	358
Qwen3-8B	63 (16.5%)	11	52	330
Gemma-4-E4B	55 (14.4%)	16	39	343

347 deployed on the first candidate, 11 deployed after repair, and 24 rejected.

Since this distribution is split by the gate's own verdict, the question to ask is not the distribution but the *accuracy of the classification*: (1) does code classified as deployable really behave as the user-approved IR, and (2) was code classified as rejected really divergent? We re-examined both directions independently.

Rejection Soundness: We audited all 35 candidates the gate flagged (9.2% of 382). The audit revealed that 33 are genuine behavioral divergences (sustain mistimings, inverted logic, counter misuse, sign-ignoring arithmetic) while 2 are subset rejections of unparseable code. Crucially, no correct program was rejected by mistake. To further validate these findings, we replayed the 22 simulatable rejections under randomized dense scenarios unrelated to the boundary synthesis. This replay reproduced the divergence in 20 cases (the remainder are of the kind that requires hitting a boundary precisely). Without the gate, the 33 behaviorally divergent candidates would have deployed while silently diverging from the approved IR.

Deployment Safety: We replayed every deployed (IR, code) pair under 100 randomized dense scenarios each, 35,800 in total, using the same trace-equivalence criteria. This re-examination confirmed that every deployed automation conforms within the bounded observation model. Specifically, 356 matched on every scenario; the remaining two differ only at the horizon edge, where a phase-late periodic lowering shifts one action group past the replay window, an artifact of the bounded horizon (§6.5), not a divergence within the active window.

Evaluation Across Generation Models: To evaluate robustness, we repeated the pipeline using two additional generators, one smaller and one from a different model family. Although they produced up to 1.8× more flagged candidates and the error rate is not monotone in model size, the classification accuracy holds. All flagged candidates are repaired or rejected, and no divergent candidate was classified as deployable (Table 5). Because the gate is deterministic, this property does not depend on generation stochasticity. Note that these metrics evaluate correctness relative to the approved IR, distinct from whether the IR itself aligns with the user's latent intent (Layer A; §9).

8.4 RQ4: On-Device Cost and Deployment

The remaining question is whether all of this is affordable on an edge device. We evaluate verification costs via latency distribution with memory and power, and close with a live deployment on a physical testbed.

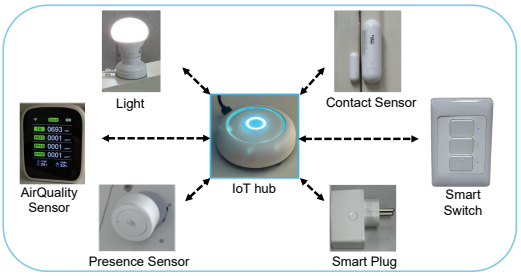


Figure 5: The physical testbed: a commercial edge hub with the sensors and actuators used in the deployment demonstration.

On-device Cost: On the M4, the verifier checks one automation with a median latency of 0.97ms (10 repetitions per automation) peaking at 54MB of memory and drawing 5.5W with no GPU use. The latency distribution is long-tailed (p95=0.7s, worst-case=8.4s). This tail originates exclusively from a specific automation class whose 1-second polling period must be stepped through tick by tick across the multi-day verification horizon, amounting to hundreds of thousands of simulated ticks, and is bounded by the simulator's tick cap. Note that LLM cost is confined to the authoring step, and a deployed automation makes zero LLM calls.

Deployment: To demonstrate feasibility, we deployed the pipeline on a commercial home-automation platform utilizing a Raspberry Pi based edge hub and physical devices (presence, contact, and air-quality sensors, plugs, lights, and a speaker; Figure 5). We authored six new commands live through the entire pipeline (IR extraction, confirmation, lowering, and verification) and registered the gate-approved lowerings. This deployment serves as an existence proof of end-to-end integration.

Table 6 summarizes the six registered automations, detailing their behavior pattern, gate outcome, and observed trace. Every observed action sequence matched the IR-predicted trace at the platform's logging resolution, including what must *not* happen (a counted cycle stopping after exactly 3 announcements; no more while a crossed threshold stays high). Figure 6 plots the observed trace of the first row, "if no one is present in the meeting room for at least 5 minutes, turn off the TV plug." The plot highlights a sharp contrast in firing times: all four ungated lowerings suffered from a sustain fault, prematurely turning off the TV just 30 seconds after the room was vacated. In contrast, the gate-repaired lowering fired precisely at 5:00, exactly as predicted by the IR. Authoring also exercised the upstream gates: two initial wordings produced a one-shot conditional IR whose rendering revealed the misreading at confirmation, and one out-of-catalog target was rejected fail-closed at extraction validation.

9 Limitations and Future Work

The primary limitation of this work is that aligning the natural-language intent with the generated IR (Layer A) remains an open challenge. If a user inadvertently approves an incorrect IR, the verifier will faithfully validate the code against that faulty specification. Assessing human confirmation behavior fell outside the scope of this study; we focused strictly on establishing that the rendering provides a deterministic, faithful plain-language representation. Accordingly, no human-participant data was collected.

The technical boundaries of the check itself, what the synthesized scenarios cannot reach and what the bounded observation model does not cover, are stated where they arise (§6.5, §8.2, §8.3). The principal open question is generalizing beyond a single target platform and DSL.

Table 6: Live-authored automations on the physical testbed: gate outcome and predicted-vs-observed behavior.

Automation	Pattern	Gate	Observed
presence absent 5min → TV plug off	sustain	repaired	off at 5:00
speak every 30s, 3 times only	count	pass	3 only, stops
door opens → light on	edge	pass	immediate
light off, on after 3min	delay	pass	on at 3:00
CO_2 rises above 900 → announce	rising	pass	on crossing only
temp. rises above 28° → switch on	rising	pass	on crossing only

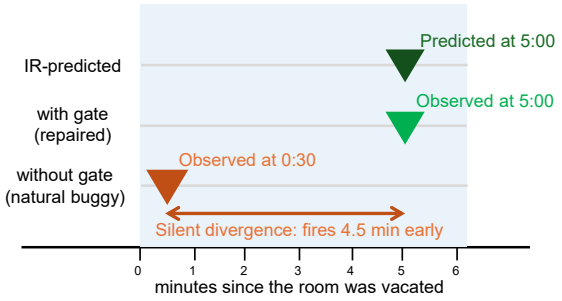


Figure 6: Observed TV-plug-off times for the first automation of Table 6: the ungated lowering fires at 0:30 after the room is vacated; the gate-repaired lowering at 5:00, as the IR predicts.

10 Conclusion

Generative LLMs have made reactive-temporal IoT automations easy to create, but verifying them before deployment has remained open for lack of a specification. OVLA derives that specification at authoring time: the system extracts a Timeline IR from the request, the user approves its deterministic plain-language rendering, and a deterministic checker then tests the generated code against the approved IR by bounded trace-equivalence, repairing or rejecting divergent code fail-closed. Across 382 automations, every candidate flagged by the gate (9.2% for the strongest generator) was repaired or rejected rather than deployed, and this classification was confirmed by a full audit of the rejections and 35,800 randomized dense replays of the deployed set. OVLA eliminates the silent divergence between the specification the user approved and the code that runs.

References

[1] Rajeev Alur and David L. Dill. 1994. A Theory of Timed Automata. *Theoretical Computer Science* 126, 2 (1994), 183–235.

[2] Z. Berkay Celik, Patrick McDaniel, and Gang Tan. 2018. Soteria: Automated IoT Safety and Security Analysis. In *Proceedings of the 2018 USENIX Annual Technical Conference* (*USENIX ATC*). USENIX Association, 147–158.

[3] Z. Berkay Celik, Gang Tan, and Patrick McDaniel. 2019. IoTGuard: Dynamic Enforcement of Security and Safety Policy in Commodity IoT. In *Network and Distributed System Security Symposium* (*NDSS*).
[4] Bei Chen, Fengji Zhang, Anh Nguyen, Daoguang Zan, Zeqi Lin, Jian-Guang Lou, and Weizhu Chen. 2023. CodeT: Code Generation with Generated Tests. In *Proceedings of the 11th International Conference on Learning Representations* (*ICLR*).
[5] Mark Chen, Jerry Tworek, Heewoo Jun, et al. 2021. Evaluating Large Language Models Trained on Code. *arXiv preprint arXiv:2107.03374* (2021).
[6] Xinyun Chen, Maxwell Lin, Nathanael Schärli, and Denny Zhou. 2024. Teaching Large Language Models to Self-Debug. In *Proceedings of the 12th International Conference on Learning Representations* (*ICLR*).
[7] Ye Cheng, Minghui Xu, Yue Zhang, Kun Li, Ruoxi Wang, and Lian Yang. 2024. AutoIoT: Automated IoT Platform Using Large Language Models. arXiv:2411.10665 [cs.SE]
[8] Ye Cheng, Minghui Xu, Yue Zhang, Kun Li, Hao Wu, Yechao Zhang, Shaoyong Guo, Wangjie Qiu, Dongxiao Yu, and Xiuzhen Cheng. 2025. Say What You Mean: Natural Language Access Control with Large Language Models for Internet of Things. arXiv:2505.23835 [cs.CR]
[9] Edmund M. Clarke, Orna Grumberg, and Doron A. Peled. 1999. *Model Checking*. MIT Press.
[10] Chenglong Fu, Qiang Zeng, and Xiaojiang Du. 2021. HAWatcher: Semantics-Aware Anomaly Detection for Appified Smart Homes. In *Proceedings of the 30th USENIX Security Symposium* (*USENIX Security*). USENIX Association, 4223–4240.
[11] Yi Gao, Kaijie Xiao, Fu Li, Weifeng Xu, Jiaming Huang, and Wei Dong. 2024. ChatIoT: Zero-code Generation of Trigger-action Based IoT Programs. *Proceedings of the ACM on Interactive, Mobile, Wearable and Ubiquitous Technologies* 8, 3, Article 103 (2024). doi:10.1145/3678585
[12] Gemma Team, Google DeepMind. 2026. Gemma 4 Model Card. https://ai.google.dev/gemma/docs/core/model_card_4.
[13] Mathyas Giudici, Alessandro Sironi, Ismaele Villa, Samuele Scherini, and Franca Garzotto. 2025. Generating Home Assistant Automations Using an LLM-based Chatbot. arXiv:2505.02802 [cs.HC]
[14] Xinxin Jin, Zhengwei Ni, Zhengguo Sheng, and Victor C. M. Leung. 2026. Proactive Rejection and Grounded Execution: A Dual-Stage Intent Analysis Paradigm for Safe and Efficient AIoT Smart Homes. arXiv:2603.16207 [cs.AI]
[15] Evan King, Haoxiang Yu, Sangsu Lee, and Christine Julien. 2024. Sasha: Creative Goal-Oriented Reasoning in Smart Homes with Large Language Models. *Proceedings of the ACM on Interactive, Mobile, Wearable and Ubiquitous Technologies* 8, 1, Article 12 (2024), 38 pages. doi:10.1145/3643505
[16] Kaiwei Liu, Bufang Yang, Lilin Xu, Yunqi Guo, Guoliang Xing, Xian Shuai, Xiaozhe Ren, Xin Jiang, and Zhenyu Yan. 2025. TaskSense: A Translation-like Approach for Tasking Heterogeneous Sensor Systems with LLMs. In *Proceedings of the 23rd ACM Conference on Embedded Networked Sensor Systems* (*SenSys*). ACM. doi:10.1145/3715014.3722070
[17] Dang Tu Nguyen, Chengyu Song, Zhiyun Qian, Srikanth V. Krishnamurthy, Edward J. M. Colbert, and Patrick McDaniel. 2018. IotSan: Fortifying the Safety of IoT Systems. In *Proceedings of the 14th International Conference on emerging Networking EXperiments and Technologies* (*CoNEXT*).
[18] OpenAI. 2025. GPT-5.1. https://openai.com.
[19] Qwen Team. 2025. Qwen3 Technical Report. arXiv:2505.09388
[20] Qwen Team. 2026. Qwen3.5-9B Model Card. https://huggingface.co/Qwen/Qwen3.5-9B.
[21] Dmitriy Rivkin, Francois Hogan, Amal Feriani, Abhisek Konar, Adam Sigal, Steve Liu, and Gregory Dudek. 2023. SAGE: Smart home Agent with Grounded Execution. arXiv:2311.00772 [cs.AI]
[22] Gyuhyeon Seo, Jungwoo Yang, Junseong Pyo, Nalim Kim, Jonggeun Lee, and Yohan Jo. 2025. SimuHome: A Temporal- and Environment-Aware Benchmark for Smart Home LLM Agents. arXiv:2509.24282 [cs.AI]
[23] Leming Shen, Qiang Yang, Xinyu Huang, Zijing Ma, and Yuanqing Zheng. 2025. GPIoT: Tailoring Small Language Models for IoT Program Synthesis and Development. In *Proceedings of the 23rd ACM Conference on Embedded Networked Sensor Systems* (*SenSys*). ACM.
[24] Leming Shen, Qiang Yang, Yuanqing Zheng, and Mo Li. 2025. AutoIOT: LLM-Driven Automated Natural Language Programming for AIoT Applications. In *Proceedings of the 31st Annual International Conference on Mobile Computing and Networking* (*MobiCom*). ACM.
[25] Yingtian Shi, Xiaoyi Liu, Chun Yu, Tianao Yang, Cheng Gao, Chen Liang, and Yuanchun Shi. 2024. Bridging the Gap Between Natural User Expression with Complex Automation Programming in Smart Homes. arXiv:2408.12687 [cs.HC]
[26] Blase Ur, Elyse McManus, Melwyn Pak Yong Ho, and Michael L. Littman. 2014. Practical Trigger-Action Programming in the Smart Home. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems* (*CHI*).
[27] Haoyu Wang, Christopher M. Poskitt, and Jun Sun. 2026. AgentSpec: Customizable Runtime Enforcement for Safe and Reliable LLM Agents. In *Proceedings of the 48th IEEE/ACM International Conference on Software Engineering* (*ICSE*). ACM.
[28] Qi Wang, Pubali Datta, Wei Yang, Si Liu, Adam Bates, and Carl A. Gunter. 2019. Charting the Attack Surface of Trigger-Action IoT Platforms. In *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security* (*CCS*). ACM, 1439–1453. doi:10.1145/3319535.3345662
[29] Chaerin Yu, Chihun Choi, Sunjae Lee, Hyosu Kim, Steven Y. Ko, Young-Bae Ko, and Sangeun Oh. 2026. Leveraging LLMs for Efficient and Personalized Smart Home Automation. arXiv:2601.04680 [cs.HC]
[30] Tao Yu, Rui Zhang, Kai Yang, Michihiro Yasunaga, Dongxu Wang, Zifan Li, James Ma, Irene Li, Qingning Yao, Shanelle Roman, Zilin Zhang, and Dragomir Radev. 2018. Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing* (*EMNLP*). Association for Computational Linguistics, 3911–3921. doi:10.18653/v1/D18-1425
[31] Yinbo Yu and Jiajia Liu. 2022. TAPInspector: Safety and Liveness Verification of Concurrent Trigger-Action IoT Systems. *IEEE Transactions on Information Forensics and Security* 17 (2022), 3773–3788. doi:10.1109/TIFS.2022.3214084
[32] Yinbo Yu, Yuanqi Xu, Kepu Huang, and Jiajia Liu. 2024. TAPFixer: Automatic Detection and Repair of Home Automation Vulnerabilities Based on Negated-property Reasoning. In *Proceedings of the 33rd USENIX Security Symposium* (*USENIX Security*). USENIX Association.
[33] Lefan Zhang, Weijia He, Jesse Martinez, Noah Brackenbury, Shan Lu, and Blase Ur. 2019. AutoTap: Synthesizing and Repairing Trigger-Action Programs Using LTL Properties. In *Proceedings of the 41st International Conference on Software Engineering* (*ICSE*). IEEE/ACM, 281–291. doi:10.1109/ICSE.2019.00043
[34] Lefan Zhang, Cyrus Zhou, Michael L. Littman, Blase Ur, and Shan Lu. 2022. Helping Users Debug Trigger-Action Programs. *Proceedings of the ACM on Interactive, Mobile, Wearable and Ubiquitous Technologies* 6, 4, Article 196 (2022), 32 pages. doi:10.1145/3569506
[35] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, et al. 2023. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems* (*NeurIPS*).

A Timeline IR Grammar

A Timeline IR is serialized as a JSON object whose steps each carry an `op` tag; the grammar below gives the abstract syntax in EBNF. Terminals are quoted; `[ x ]` marks an optional part and `{ x }` zero-or-more repetition.

```
ir       ::= start_at { step }
step     ::= wait | delay | read | call
           | if | cycle | break

start_at ::= "start_at" "(" ("now" | "cron" cron) ")"
wait     ::= "wait" "(" expr ["," "edge:" edge]
               ["," "for:" duration] ")"
edge     ::= "none" | "rising" | "falling"
delay    ::= "delay" "(" duration ")"
read     ::= "read" "(" ident "," service "." attr ")"
call     ::= "call" "(" service "." function
               { "," name "=" (value | expr) } ")"
if       ::= "if" "(" expr ")" "{" { step } "}"
               ["else" "{" { step } "}"]
cycle    ::= "cycle" "(" "period:" duration
               ["," "count:" ident]
               ["," "until:" expr] ")"
               "{" step { step } "}"
break    ::= "break"

duration ::= int ("HOUR" | "MIN" | "SEC" | "MSEC")

expr     ::= expr ("or" | "and") expr | "not" expr
           | arith [cmp arith]
cmp      ::= "==" | "!=" | "<" | "<=" | ">" | ">="
arith    ::= arith ("+" | "-" | "*" | "/") arith
           | "abs" "(" expr ")"
           | ("min" | "max") "(" expr "," expr ")"
           | service "." attr | "$" ident | literal
```

`service.attr` denotes a live sensor read and `$ident` a persistent variable bound by `read` or `cycle.count`; `literal` ranges over

numbers, booleans, and the enum values declared in the device catalog.

Semantic conventions. `wait.edge="rising"` completes once per upward crossing of `cond`; `wait.for` requires `cond` to hold continuously for the duration, resetting if it flips false. `cycle.period` is the per-iteration cadence; when `until` is absent, the cycle repeats within the verifier's bounded horizon (§6).

Context-sensitive rules (*grammar G, §5*). Beyond per-step shape, the feasibility gate enforces: (1) `timeline[0]` is the only `start_at`; (2) `start_at` may not appear inside a branch or loop body; (3) at most one top-level `cycle` and no nested `cycle`; (4) `break` only inside a `cycle` body; (5) `cycle.period` is required. Catalog conformance (every `Service.Attr` and `call.target` names a service, member, and argument that the connected-device catalog declares) is checked at the same point and rejects out-of-catalog targets fail-closed.

B Dataset Category Breakdown

Table 7 reports the 24-category breakdown of the 382-command benchmark. Each category corresponds to a structurally distinct reactive pattern and its lowering idiom.

Table 7: Dataset categories (382 commands, 24 categories).

ID	Pattern	n
C01	Single device action	28
C02	Action with device selector	33
C03	Single-condition `if` (incl. if-else)	34
C05	AND-compound `if`	30
C06	OR-compound / chained `if-elif-else`	7
C07	Level-triggered wait ("when X, do Y")	32
C08	Rising-edge trigger ("whenever")	41
C09	Sequential delay	18
C10	Trigger + delayed action	9
C11	Snapshot, delay, and re-read comparison	8
C12	Phase lifecycle ("when X, every N thereafter")	15
C13	Alternation / binary toggle	7
C14	Progressive update with counter and stop	7
C15	Cron-scheduled action	21
C16	Cron + branch	13
C17	Periodic polling + branch	12
C18	Bounded duration (time-window cycle)	10
C19	Hysteresis (double-threshold deadband)	8
C20	Sustained condition ("for at least N")	17
C21	Group consensus (all/any quantifier)	7
C22	Counted repetition ("repeat N times, stop")	10
C23	Sustained trigger + multi-step sequence	5
C24	Sustained trigger + repeating alert	5
C25	Periodic polling + if-else	5
	Total	382